



Rapport-SAE 4.DEVCLLOUD.01

Application de gestion bancaire en micro-services

Lien GitHub du groupe : https://github.com/LionelG-hub/Sae-401Application_de_gestion_bancaire

1. Présentation du projet

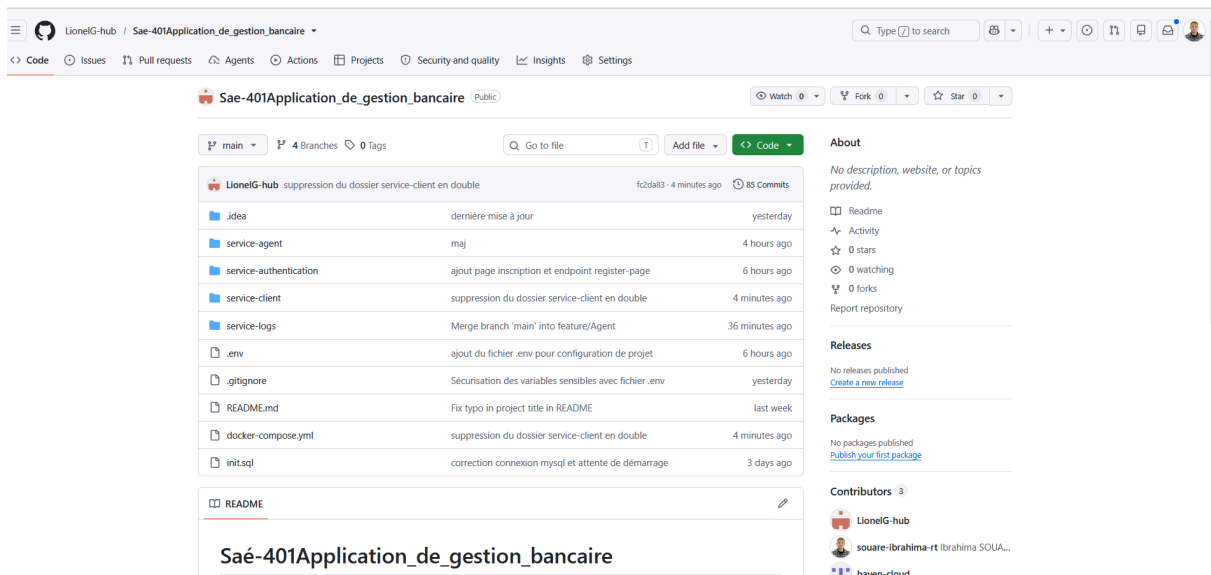
Ce projet consiste à concevoir une application de gestion bancaire développée sous forme de micro-services. L'objectif est de mettre en place une architecture distribuée où chaque service est indépendant et communique avec les autres via des API REST.

Le projet a été réalisé en groupe de trois étudiants. Chacun était responsable d'un service distinct :

Ibrahima Souaré : Service Authentification

Lionel : Service Agent Bancaire

Baye : Service Client



2. Architecture du projet

L'application est composée de six éléments qui fonctionnent ensemble via Docker Compose : Le service authentification gère la connexion des utilisateurs et la génération des tokens JWT. Il est accessible sur le port 8000.

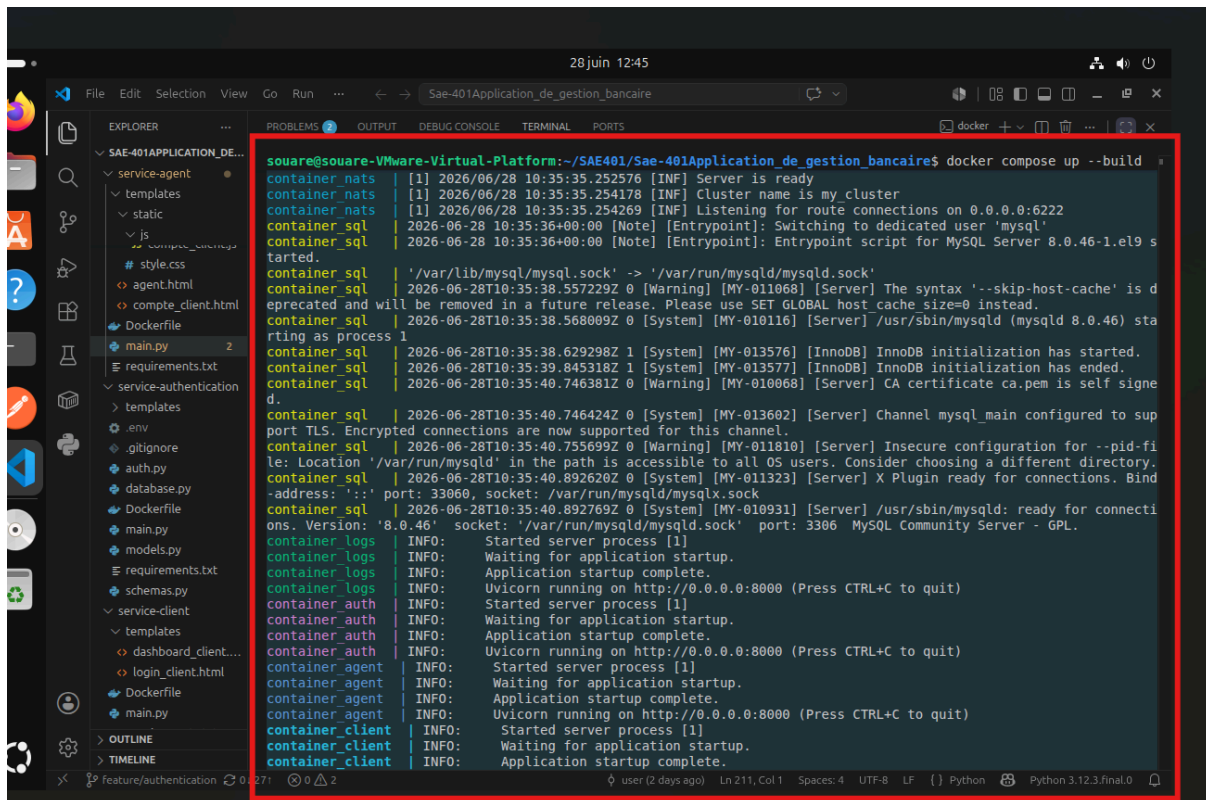
Le service client permet aux clients de consulter leurs comptes et d'effectuer des opérations bancaires. Il tourne sur le port 8001.

Le service agent bancaire permet aux agents de valider ou refuser les opérations des clients. Il est disponible sur le port 8004.

Le service logs collecte tous les événements publiés par les autres services via NATS et les stocke en base de données. Il tourne sur le port 8005.

NATS est le serveur de messagerie qui permet aux services de publier et recevoir des messages de façon asynchrone. Il est accessible sur le port 4222.

La base de données MySQL est partagée entre les services avec une base de données dédiée par service : auth_db pour l'authentification, bank_db pour les opérations bancaires et logs_db pour les logs.



3. Mon service : Service Authentication

3.1 Rôle du service

Mon service est la porte d'entrée de toute l'application. Il est responsable de l'authentification des utilisateurs, de la génération des tokens JWT et de la limitation des accès aux interfaces web et aux API. Sans mon service, aucun autre service ne peut fonctionner car ils dépendent tous de la vérification des tokens JWT.

3.2 Technologies utilisées

FastAPI est le framework Python choisi pour créer les API. Il est moderne, rapide et génère automatiquement la documentation de l'API.

SQLAlchemy est la bibliothèque utilisée pour communiquer avec MySQL. Elle permet de définir les modèles de données en Python sans écrire de SQL directement.

MySQL est la base de données relationnelle utilisée pour stocker les utilisateurs. Elle tourne dans un conteneur Docker dédié.

JWT (JSON Web Token) est le standard utilisé pour l'authentification. Après connexion, chaque utilisateur reçoit un token signé qui contient son identité et son rôle.

Passlib avec bcrypt est utilisé pour hasher les mots de passe avant de les stocker en base de données. Le vrai mot de passe n'est jamais stocké.

NATS est le serveur de messagerie utilisé pour publier les logs d'authentification.

Docker permet de conteneuriser le service et de le lancer de façon isolée.

Jinja2 est le moteur de templates utilisé pour servir les pages HTML depuis FastAPI.

3.3 Structure du service

Le service est organisé en plusieurs fichiers, chacun ayant un rôle précis :

Le fichier main.py contient tous les endpoints de l'API et la configuration de l'application FastAPI.

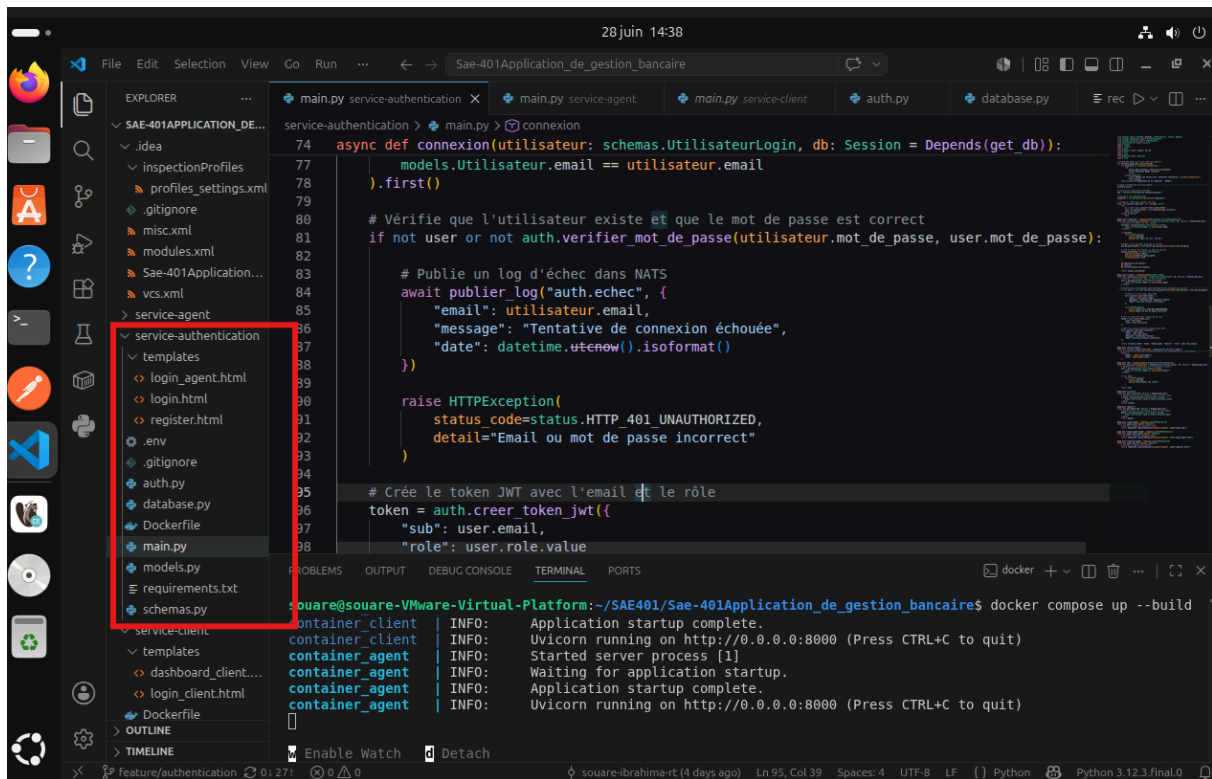
Le fichier auth.py contient toute la logique JWT : génération des tokens, vérification, hashage des mots de passe et gestion des rôles.

Le fichier models.py définit la structure de la table utilisateurs dans MySQL avec SQLAlchemy.

Le fichier schemas.py définit les schémas de validation des données entrantes et sortantes avec Pydantic.

Le fichier database.py gère la connexion à MySQL et fournit les sessions de base de données aux endpoints.

Le dossier templates contient les pages HTML : login.html pour la connexion client, login_agent.html pour la connexion agent et register.html pour l'inscription.

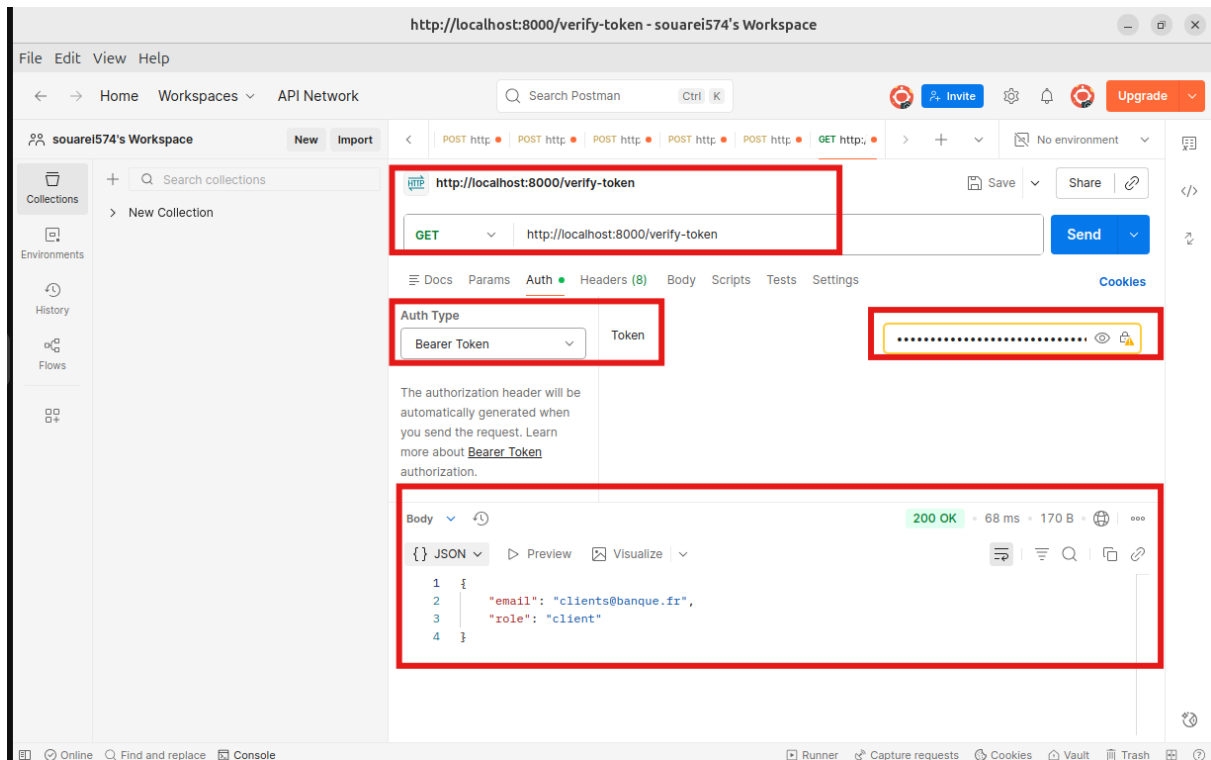


3.4 Les endpoints développés

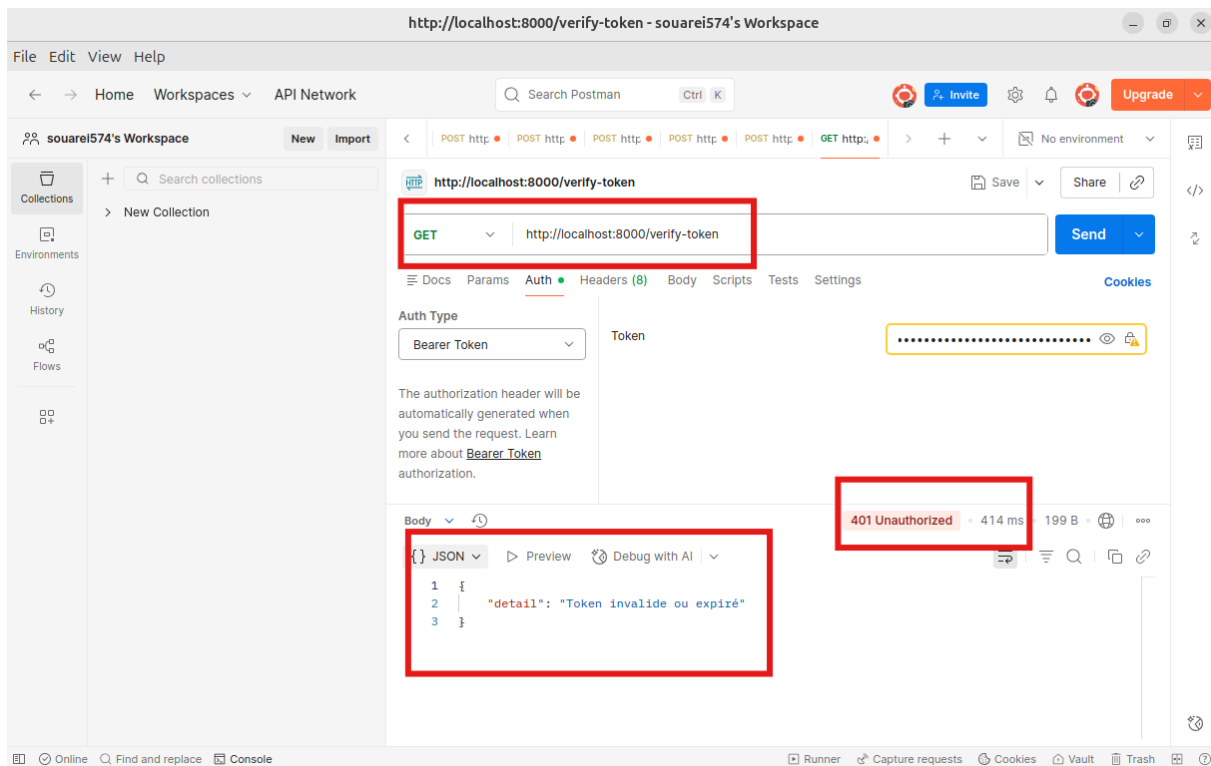
J'ai développé les endpoints suivants :

POST /register permet de créer un nouveau compte utilisateur avec un email, un mot de passe et un rôle. Le mot de passe est automatiquement hashé avant d'être stocké.

authentifié.



Et là je met une mauvais token pour voir



GET /me retourne les informations de l'utilisateur actuellement connecté en décodant son token JWT.

GET /clients retourne la liste de tous les utilisateurs ayant le rôle client. Cet endpoint est utilisé par le service agent pour afficher la liste des clients.

GET /agents retourne la liste de tous les utilisateurs ayant le rôle agent.

GET /login-page sert la page HTML de connexion pour les clients.

GET /login-agent-page sert la page HTML de connexion pour les agents bancaires.

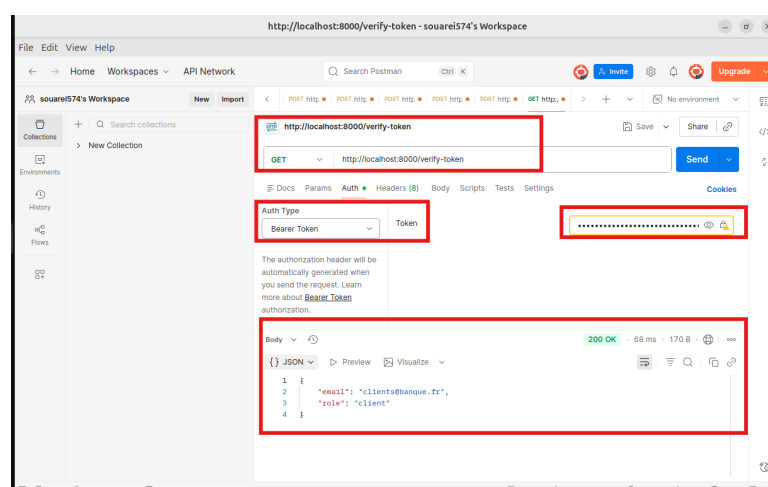
GET /register-page sert la page HTML d'inscription pour créer un nouveau compte.

3.5 Fonctionnement du JWT

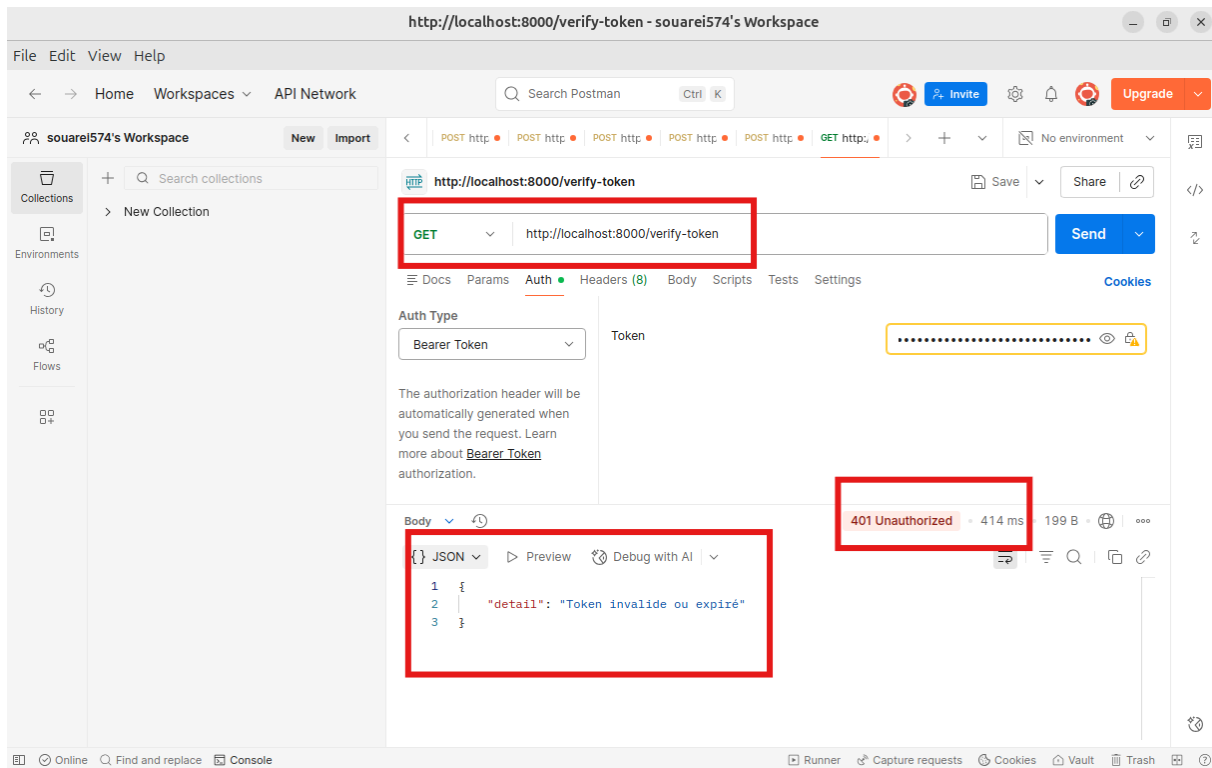
Quand un utilisateur se connecte, le service génère un token JWT signé avec une clé secrète. Ce token contient l'email de l'utilisateur, son rôle et une date d'expiration de 30 minutes. Le token est retourné au client qui le stocke dans le **sessionStorage** du navigateur.

À chaque requête vers les autres services, le token est envoyé dans le header HTTP. Le service appelé vérifie le token en appelant mon endpoint GET /verify-token. Si le token est valide et que le rôle correspond, l'accès est autorisé. Sinon, une erreur 401 est retournée.

vérification avec une bonne token



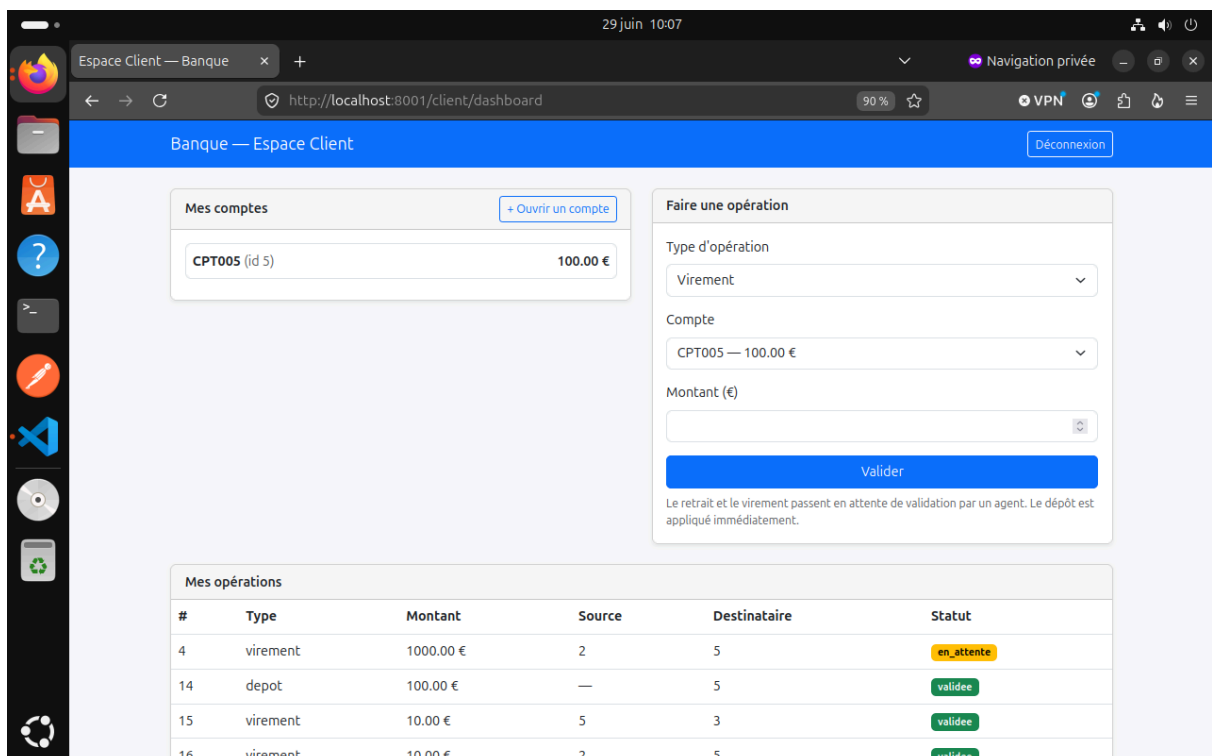
vérification avec une mauvais token



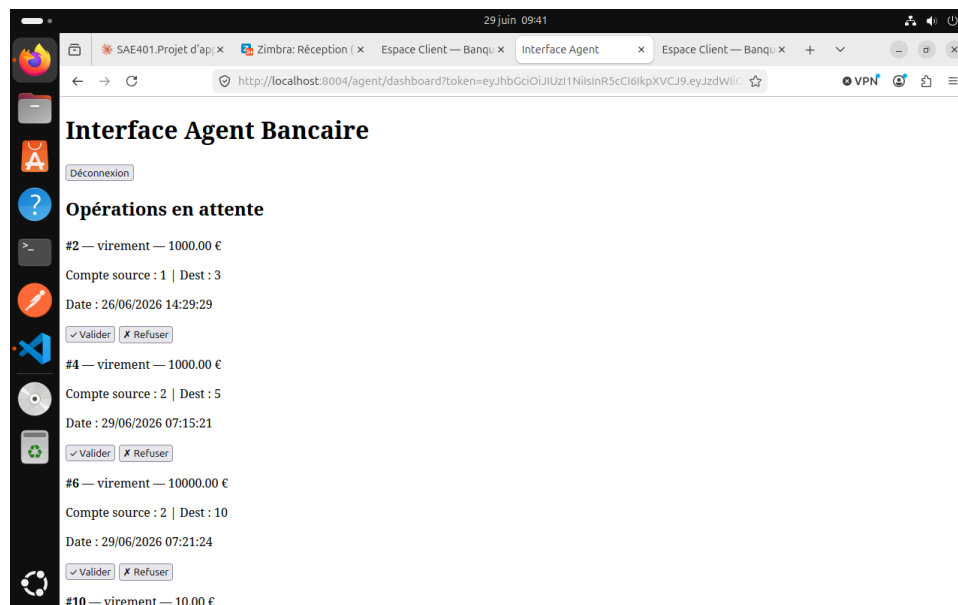
3.6 Gestion des rôles

L'application distingue deux types d'utilisateurs avec des droits différents.

Le client peut consulter ses comptes, faire des dépôts, des retraits et des virements. Il n'a pas accès aux fonctionnalités de l'agent bancaire.

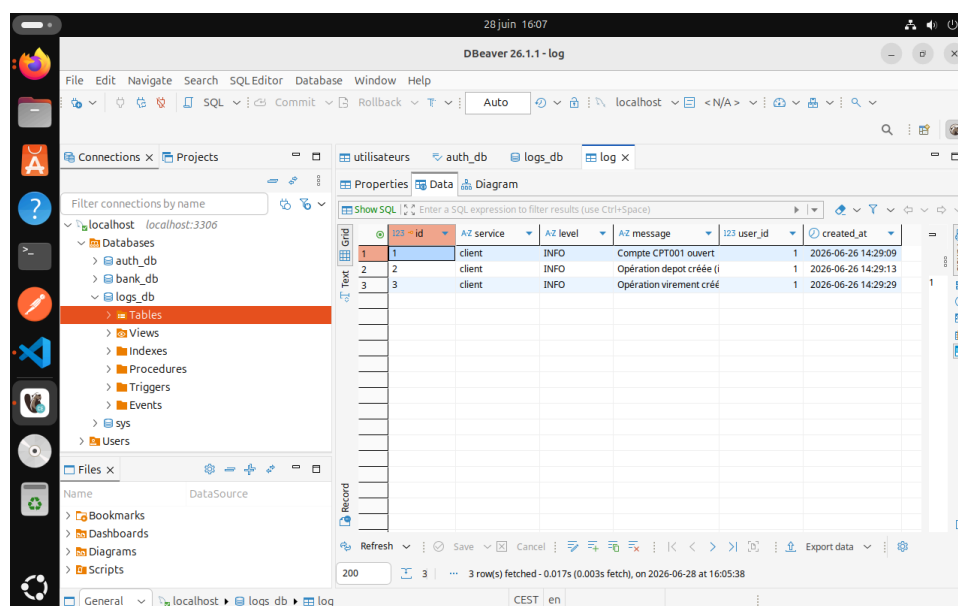


L'agent bancaire peut voir la liste des opérations en attente, les valider ou les refuser et consulter les comptes des clients. Il n'a pas accès aux fonctionnalités du client.



3.7 Publication des logs dans NATS

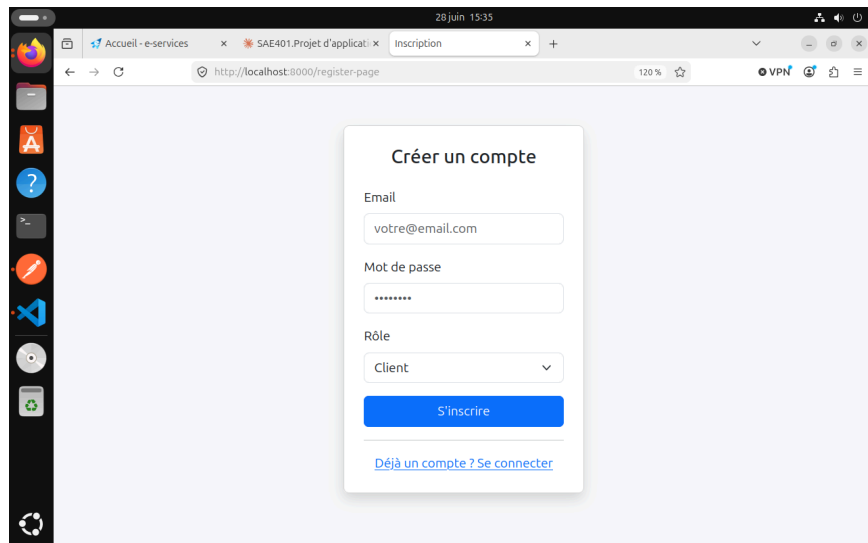
À chaque événement d'authentification, mon service publie un message dans NATS. Une connexion réussie publie un log sur le sujet `auth.connexion` avec l'email, le rôle et la date. Un échec de connexion publie un log sur le sujet `auth.echec`. Le service logs est abonné à ces sujets et stocke les logs en base de données.



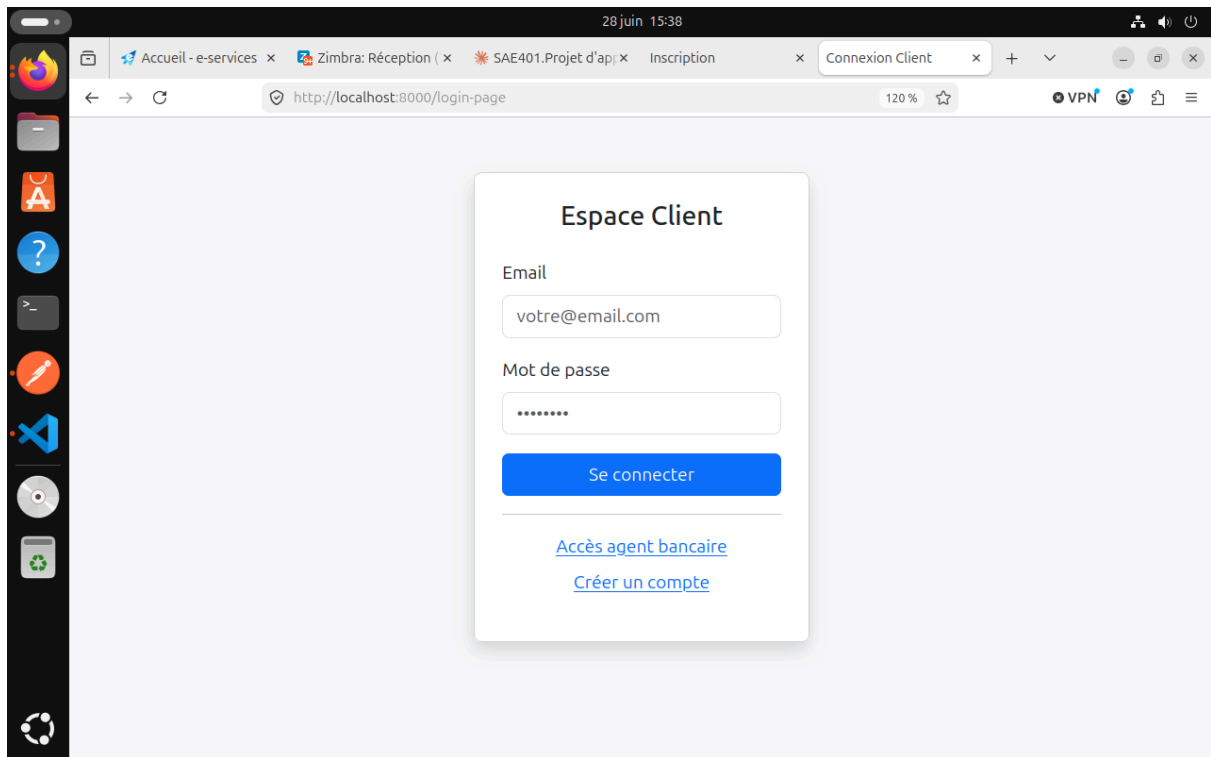
3.8 Interface web

J'ai développé trois pages HTML pour l'interface web du service :

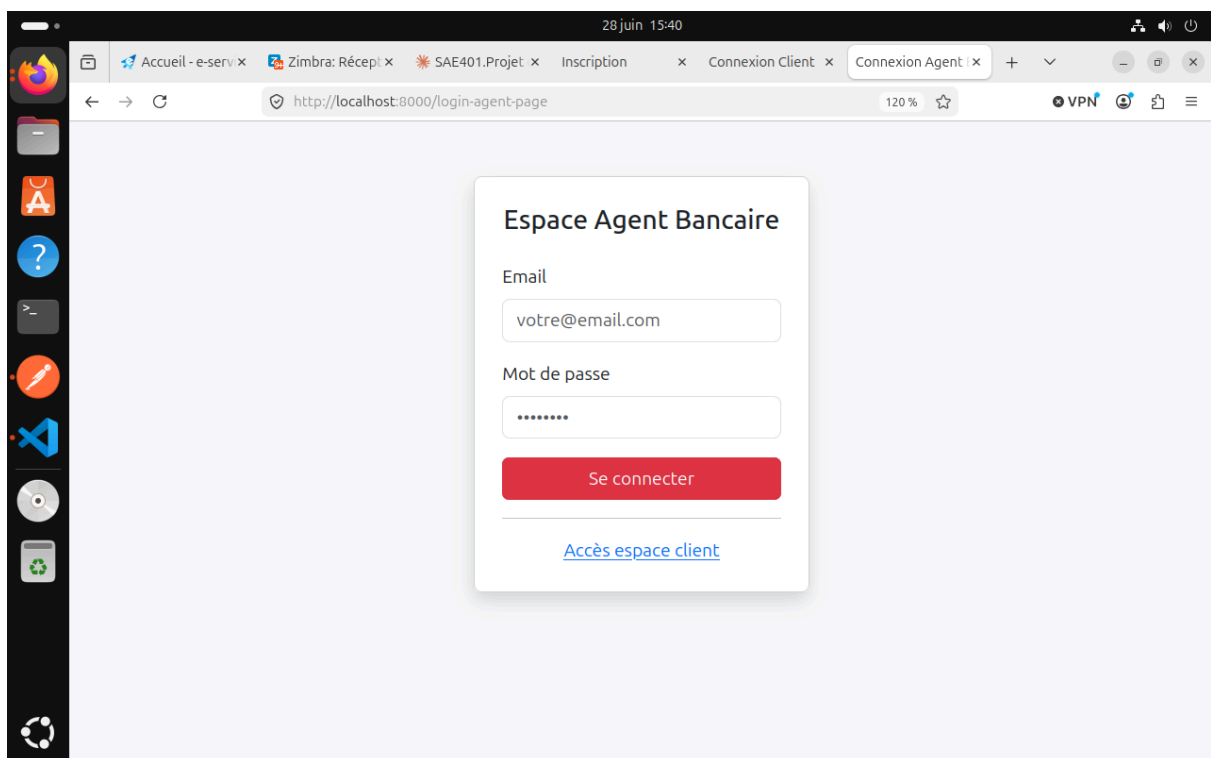
La page d'inscription permet de créer un compte avec un email, un mot de passe et un choix de rôle. Après inscription réussie, l'utilisateur est automatiquement redirigé vers la bonne page de connexion selon son rôle.



La page de connexion client permet aux clients de se connecter. Après connexion réussie, le token JWT est stocké dans le localStorage et l'utilisateur est redirigé vers le dashboard client sur le port 8001.



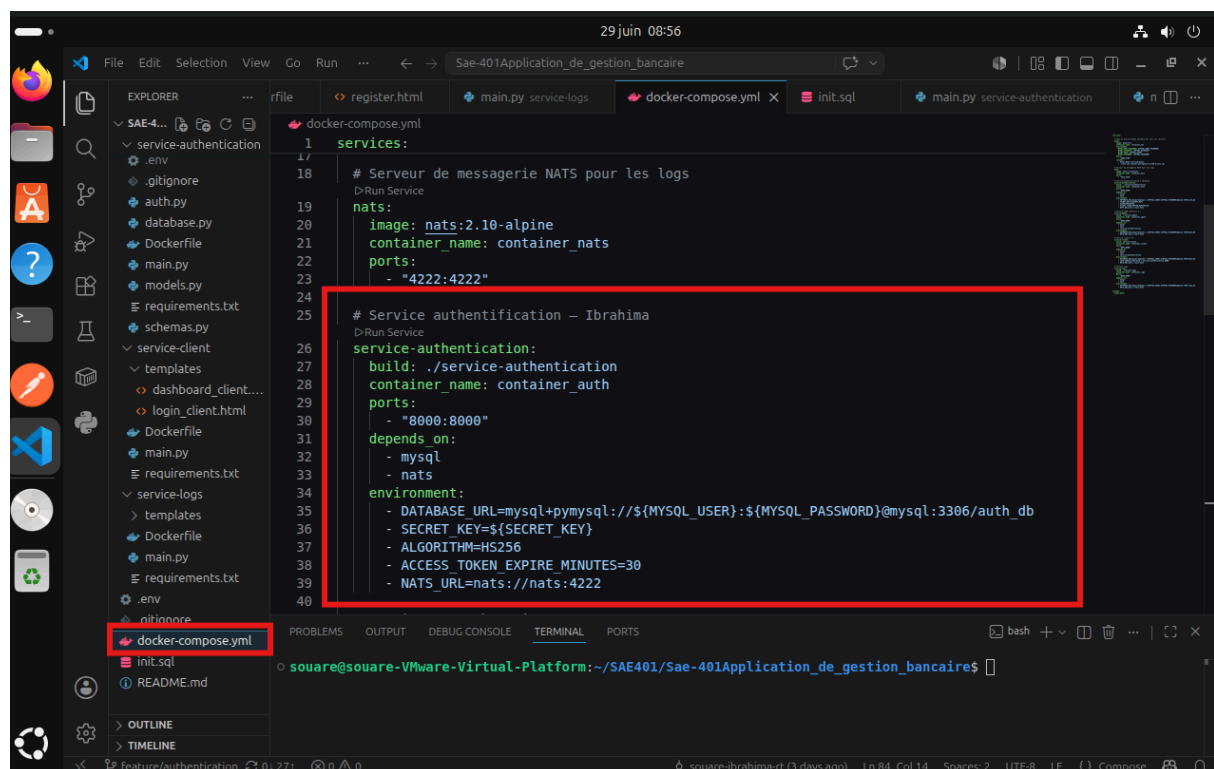
La page de connexion agent permet aux agents bancaires de se connecter. Après connexion réussie, l'agent est redirigé vers son dashboard sur le port 8004. La page vérifie que le rôle retourné est bien agent avant la redirection.



3.9 Configuration Docker

Le service est conteneurisé avec Docker. Le Dockerfile part d'une image Python 3.12-slim, installe les dépendances depuis requirements.txt et expose le port 8000. La commande de démarrage lance uvicorn qui est le serveur ASGI de FastAPI.

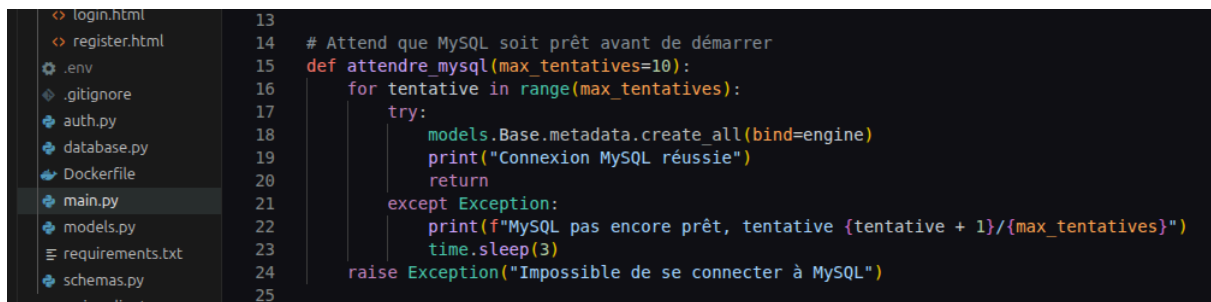
Le service est configuré dans le docker-compose.yml avec ses variables d'environnement. Il dépend de MySQL et NATS qui doivent démarrer avant lui. Une fonction attendre_mysql permet au service d'attendre que MySQL soit prêt avant de créer les tables.



```
1 services:
2   # Serveur de messagerie NATS pour les logs
3   nats:
4     image: nats:2.10-alpine
5     container_name: container_nats
6     ports:
7       - "4222:4222"
8
9   # Service authentication - Ibrahima
10  service-authentication:
11    build: ./service-authentication
12    container_name: container_auth
13    ports:
14      - "8000:8000"
15    depends_on:
16      - mysql
17      - nats
18    environment:
19      - DATABASE_URL=mysql+pymysql://${MYSQL_USER}:${MYSQL_PASSWORD}@mysql:3306/auth_db
20      - SECRET_KEY=${SECRET_KEY}
21      - ALGORITHM=HS256
22      - ACCESS_TOKEN_EXPIRE_MINUTES=30
23      - NATS_URL=nats://nats:4222
24
25  # Service principal
26  main:
27    build: .
28    container_name: container_main
29    ports:
30      - "8000:8000"
31    depends_on:
32      - mysql
33      - nats
34    environment:
35      - DATABASE_URL=mysql+pymysql://${MYSQL_USER}:${MYSQL_PASSWORD}@mysql:3306/main_db
36      - SECRET_KEY=${SECRET_KEY}
37      - ALGORITHM=HS256
38      - ACCESS_TOKEN_EXPIRE_MINUTES=30
39      - NATS_URL=nats://nats:4222
```

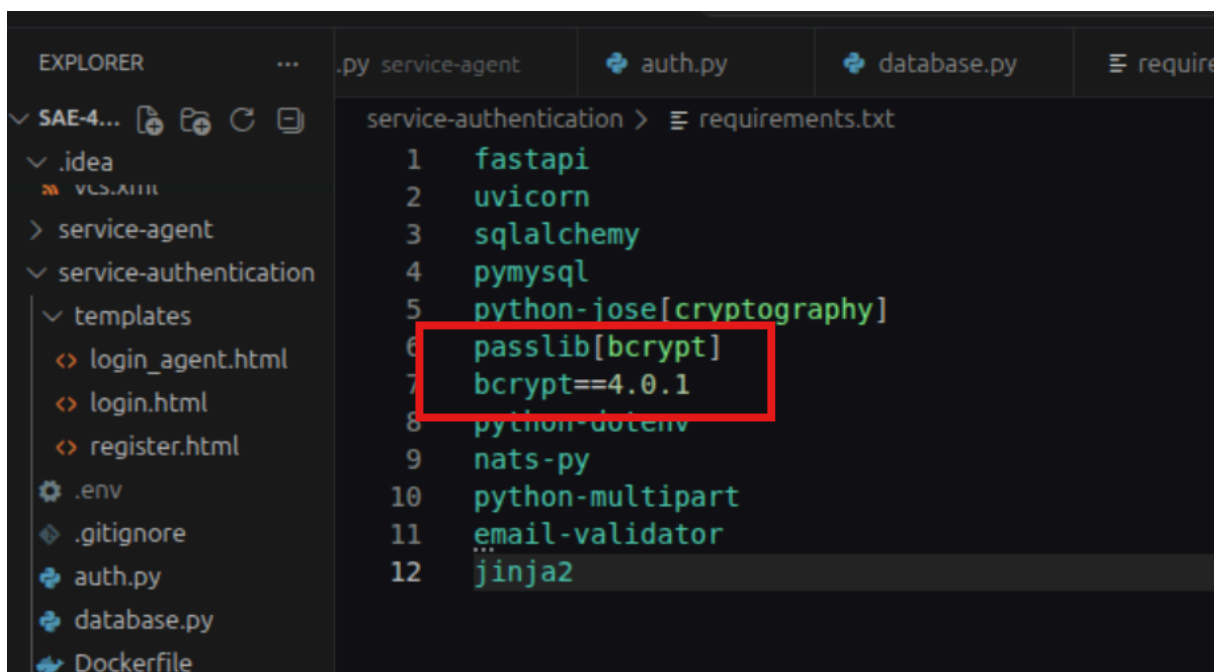
4. Difficultés rencontrées et solutions

La première difficulté rencontrée était la gestion du démarrage des services avec Docker Compose. MySQL prend quelques secondes pour démarrer complètement et mon service essayait de se connecter avant que MySQL soit prêt. J'ai résolu ce problème en implémentant une fonction attendre_mysql qui réessaie la connexion jusqu'à dix fois avec un délai de trois secondes entre chaque tentative.



```
13
14 # Attend que MySQL soit prêt avant de démarrer
15 def attendre_mysql(max_tentatives=10):
16     for tentative in range(max_tentatives):
17         try:
18             models.Base.metadata.create_all(bind=engine)
19             print("Connexion MySQL réussie")
20             return
21         except Exception:
22             print(f"MySQL pas encore prêt, tentative {tentative + 1}/{max_tentatives}")
23             time.sleep(3)
24     raise Exception("Impossible de se connecter à MySQL")
25
```

La deuxième difficulté était la compatibilité entre passlib et les nouvelles versions de bcrypt. J'ai résolu ce problème en fixant la version de bcrypt à 4.0.1 dans le fichier requirements.txt, ce qui est une bonne pratique pour garantir la stabilité du projet.



```
1 fastapi
2 uvicorn
3 sqlalchemy
4 pymysql
5 python-jose[cryptography]
6 passlib[bcrypt]
7 bcrypt==4.0.1
8 python-dotenv
9 nats-py
10 python-multipart
11 email-validator
12 jinja2
```

La troisième difficulté était la gestion des conflits Git lors des fusions entre les branches du groupe. Chacun travaillait sur sa propre branche et des conflits apparaissaient sur le fichier docker-compose.yml. J'ai résolu ces conflits manuellement en comparant les versions et en gardant le meilleur des deux.

La quatrième difficulté était la sécurité des sessions entre plusieurs clients connectés simultanément sur le même navigateur. En utilisant localStorage, le token d'un client écrasait celui d'un autre client connecté dans un autre onglet. J'ai résolu ce problème en remplaçant localStorage par sessionStorage dans le service client, ce qui isole chaque session par onglet.

```

    async function seConnecter() {
    }

    const donnees = await reponse.json();
    // Login et dashboard sur la MÊME origine (:8001) -> le
    sessionStorage.setItem('token', donnees.access_token);
    window.location.href = '/client/dashboard';
  } catch (err) {
    afficherErreur('Serveur injoignable, réessayez');
  }
}

// Permet de valider avec la touche Entrée
document.addEventListener('keydown', function (e) {
  if (e.key === 'Enter') seConnecter();
});

```

5. Compétences acquises

Ce projet m'a permis de développer et valider plusieurs compétences du parcours Développement Système et Cloud.

En ce qui concerne le développement d'applications, j'ai appris à concevoir et développer une API REST avec FastAPI en Python, à implémenter un système d'authentification sécurisé avec JWT, à gérer les rôles et les permissions dans une application web et à créer des interfaces web en HTML, CSS et JavaScript vanilla.

En ce qui concerne les micro-services et le cloud, j'ai appris à concevoir une architecture micro-services où chaque service a une responsabilité unique, à faire communiquer des services entre eux via des API REST et un système de messagerie NATS, à conteneuriser des applications avec Docker et à orchestrer plusieurs services avec Docker Compose.

En ce qui concerne la sécurité, j'ai appris à hasher les mots de passe avec bcrypt, à signer et vérifier des tokens JWT, à protéger les routes API selon les rôles et à utiliser des variables d'environnement pour ne pas exposer les informations sensibles dans le code.

En ce qui concerne le travail en équipe, j'ai appris à travailler avec Git en utilisant des branches par fonctionnalité, à faire des Pull Requests et à résoudre des conflits de fusion.

6. Conclusion

Ce projet m'a permis de mettre en pratique les concepts vus en cours sur les micro-services, la conteneurisation avec Docker et le développement d'API REST. Mon service authentification est le fondement de toute l'application sans lui aucun autre service ne peut fonctionner. J'ai particulièrement apprécié la mise en place du système JWT et la gestion des rôles qui sont des concepts fondamentaux utilisés dans toutes les applications web modernes.